

TD/TP 5

Informatique 2 : Algorithmique 2 / Python MIP - S2

Exercice 1 :

Pour chacun des algorithmes de tri suivants, suivez et notez le changement de la variable « **arr** »

```
def tri_bulles(arr):  
    n = len(arr)  
  
    for i in range(n):  
        # Les derniers i éléments seront déjà à leur place, pas besoin de les vérifier  
        for j in range(0, n - i - 1):  
            # Parcourez le tableau de 0 à n-i-1  
            # Échangez si l'élément trouvé est supérieur à l'élément suivant  
            if arr[j] > arr[j + 1]:  
                arr[j], arr[j + 1] = arr[j + 1], arr[j]  
  
    return arr  
  
a = [12, 11, 13, 5, 6]  
a_trié = tri_bulles(a)  
a_trié  
  
[5, 6, 11, 12, 13]
```

```
def tri_insertion(arr):  
    for i in range(1, len(arr)):  
        j = i  
  
        while arr[j-1] > arr[j] and j > 0:  
            arr[j-1], arr[j] = arr[j], arr[j-1]  
            j -= 1  
  
    return arr  
  
a = [22, 31, 13, 15, 1]  
a_trié = tri_insertion(a)  
print(a_trié)  
  
[1, 13, 15, 22, 31]
```

```
def tri_selection(arr):
    n = len(arr)

    for i in range(n - 1):
        # Trouver l'élément minimum dans la partie non triée du tableau
        indice_min = i
        for j in range(i + 1, n):
            if arr[j] < arr[indice_min]:
                indice_min = j

        # Échanger l'élément minimum trouvé avec le premier élément
        arr[i], arr[indice_min] = arr[indice_min], arr[i]

    return arr

a = [10, 7, 17, 25, 2]
a_trié = tri_selection(a)
print(a_trié)

[2, 7, 10, 17, 25]
```

Exercice 2 :

- Pour chaque itération, donnez les valeurs des variables « bas » et « haut »
- Quel est la condition pour que cet algorithme fonctionne bien ?

```
def recherche_binaire(arr, cible):
    bas, haut = 0, len(arr) - 1
    while bas <= haut:
        milieu = (bas + haut) // 2
        if cible == arr[milieu]:
            return milieu # Indice de l'élément cible
        elif cible > arr[milieu]:
            bas = milieu + 1
        else:
            ## cible < arr[milieu]
            haut = milieu - 1
    return -1 # Cible non trouvée

a = [3, 7, 9, 10, 12, 16, 20, 32]
res = recherche_binaire(a, 20)
res
```

6